De la maquette à une représentation hiérarchique

Étude comparative de quatre approches de génération de code frontend

Version enrichie intégrant les remarques de l'encadreur

KOUONGA TADIE KENNEDY

Université de Douala

Version révisée

Plan de la présentation

Partie A — Contenu déjà validé par l'encadreur

La question de recherche, les 4 approches, le principe commun, la question ouverte finale.

Partie B — Corrections apportées suite aux remarques

Entrées différentes, métriques chiffrées manquantes, nuance sur l'auto-correction, aspects industriels.

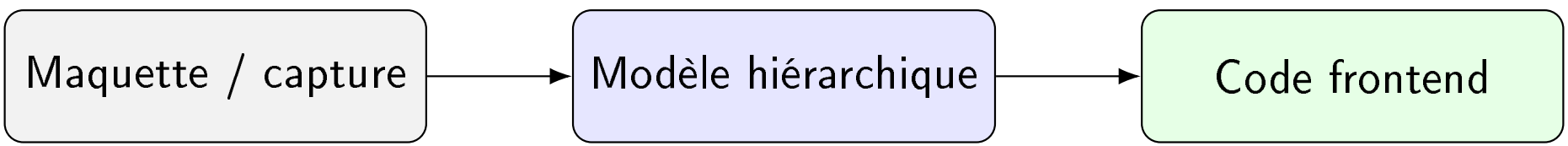
Partie C — Manques réels identifiés dans la littérature

Interactivité, accessibilité, indépendance technologique — pistes pour la suite.

Question de recherche

Problème étudié

Comment passer d'une représentation visuelle d'une interface (maquette, capture d'écran, croquis) à une **représentation structurée et hiérarchique** qui pourra ensuite servir à générer automatiquement du code frontend ?



Pourquoi une représentation intermédiaire ?

- Une image décrit surtout **ce que l'on voit**.
- Le code doit exprimer **comment les éléments sont organisés**.
- Il faut donc retrouver les éléments, leurs relations, leur organisation spatiale et leurs comportements de mise en page.

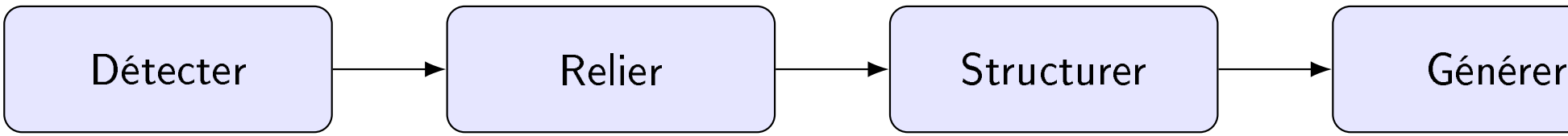
Point important

« Représentation hiérarchique » est utilisée au sens large : arbre de composants, arbre de mise en page, arbre de subdivisions ou structure avec contraintes.

Les quatre approches étudiées

Approche	**Idée principale**
Prototype2Code	Reconstruire une hiérarchie à partir des relations de contenance, puis organiser en lignes/colonnes
DCGen	Diviser récursivement l'image en segments ; la division forme un arbre
ScreenCoder	Détecter et nommer les grandes régions, puis construire un arbre selon des règles de conception web
DesignCoder	Diviser l'interface, extraire le sens des éléments, puis les regrouper dans un arbre de composants

Ce que les quatre méthodes ont en commun



1. Identifier des unités de l'interface (éléments, régions, widgets)
2. Déterminer des relations entre ces unités
3. Construire une structure (ne plus traiter l'image comme plate)
4. Utiliser cette structure pour générer le code

Question qui se dégage (validée par l'encadreur)

Excellente problématisation finale

Quel niveau d'information doit contenir notre modèle pour être indépendant de la technologie cible tout en restant suffisamment riche pour générer ensuite React, HTML/CSS, Flutter, etc. ?

L'encadreur qualifie cette question d'« ouverture scientifique très solide ».
Elle reste, comme on le verra, encore sans réponse dans la littérature (Partie C).

Remarque de l'encadreur : les entrées ne sont pas comparables

Limite soulevée

« Les 4 approches ne travaillent pas sur les mêmes entrées. Comparer sans marquer cette frontière fausse le jugement. »

Avec métadonnées de conception

- Prototype2Code
- DesignCoder

Image + fichier Figma/Sketch (id, type, bbox, style)

Image seule

- DCGen
- ScreenCoder

Aucune donnée cachée, tout est deviné

Analogie : monter un meuble avec la notice, contre le monter seulement avec la photo du résultat.

Remarque de l'encadreur : absence de métriques

Limite soulevée

« L'étude est purement qualitative. Il manque une grille de métriques concrètes (IoU, CLIP-Score, Tree Edit Distance...). »

Ce que contiennent réellement les 4 articles sources

Méthode	**Score de ressemblance visuelle (CLIP)**
ScreenCoder	81 %
DesignCoder	92 %
DCGen	86 %
Prototype2Code	non publié dans le document source

Ces chiffres existent déjà dans la littérature étudiée — ils manquaient seulement à la synthèse initiale.

La structure compte autant que l'apparence

Le vrai ajout demandé par l'encadreur

Une image générée peut sembler identique à l'originale, sans que le code soit bien organisé.

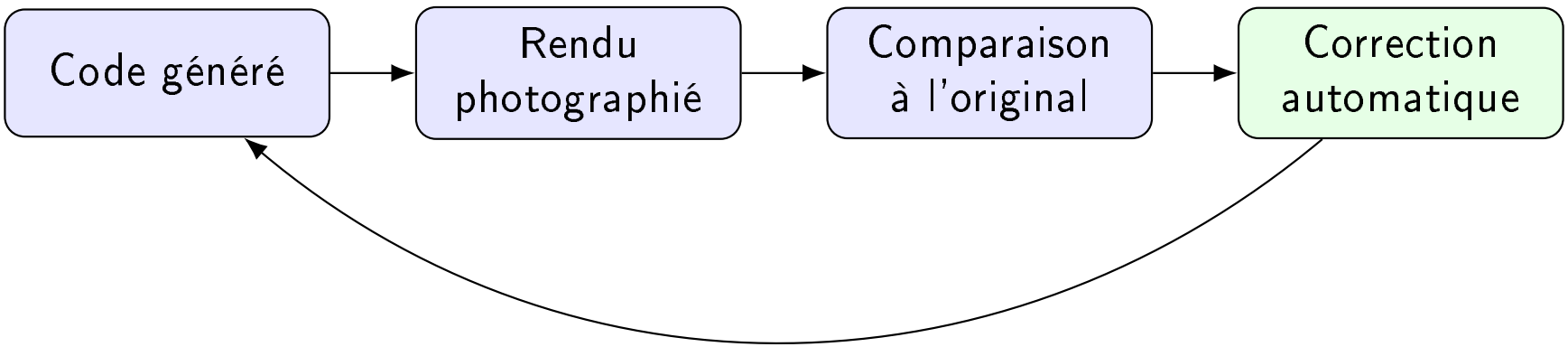
Méthode	TreeBLEU	Container Match	Tree Edit Distance
Prototype2Code	0.53	0.58	40.0 (plus bas = mieux)
DesignCoder	**0.69**	**0.75**	**28.2**

Image : deux armoires identiques fermées — ouvertes, l'une range par catégorie, l'autre entasse tout en vrac. DesignCoder range mieux à l'intérieur.

Correction factuelle : l'auto-correction existe déjà

Nuance à apporter

L'encadreur évoque un flux « unidirectionnel » (maquette → arbre → code) comme limite générale. **C'est vrai pour 3 méthodes sur 4**, mais DesignCoder possède déjà ce mécanisme.



Sans ce module, les scores de DesignCoder chutent fortement (CLIP : 0.92 → 0.88 ; TreeBLEU : 0.66 → 0.55).

Aspects industriels : partiellement couverts

Limite soulevée

« Design tokens, bibliothèques de composants, coûts d'inférence, latence ne sont pas abordés. »

Ce qui est en fait déjà présent

- ScreenCoder génère en **Tailwind CSS**, une bibliothèque de styles standard — pas du CSS fait main.
- ScreenCoder discute explicitement le compromis **coût vs qualité** et le temps de génération (« tens of seconds »).

Ce qui reste vraiment absent

Aucune méthode ne gère des design systems personnalisés (tokens de marque, variables de couleur propriétaires).

Manque confirmé A — Statique contre dynamique

Constat après lecture complète des 4 articles

Toutes les méthodes construisent un arbre à un instant T, figé.

- Aucune ne gère le survol (*hover*), les menus déroulants, les modales.
- Aucune ne gère l'adaptation à différentes tailles d'écran au-delà de la mise en page statique.

Ce point du feedback est entièrement confirmé — aucune nuance à apporter ici.

Manque confirmé B — Accessibilité

Constat après lecture complète des 4 articles

Le mot « accessibilité » (au sens ARIA, lecteurs d'écran, navigation clavier) n'apparaît dans aucun des quatre documents sources.

- Pas de rôles sémantiques HTML5 profonds (`main`, `nav`, `article`).
- Pas de prise en compte des lecteurs d'écran.

C'est un angle mort partagé par toute la littérature étudiée, pas seulement par cette synthèse.

Manque confirmé C — Dépendance à un langage cible

Constat après lecture complète des 4 articles

Chaque méthode est mariée à une seule technologie de sortie.

Méthode	**Technologie cible unique**
DesignCoder	React Native uniquement
Prototype2Code, DCGen, ScreenCoder	HTML/CSS uniquement

Aucune méthode ne produit un format intermédiaire réellement indépendant de la technologie — la question posée en Partie A reste donc entièrement ouverte.

Conclusion — Pistes pour la suite

Synthèse

Les travaux étudiés convergent vers une même idée : la maquette doit être transformée en une représentation structurée avant de produire le code. Mais des manques subsistent, confirmés par une lecture complète des sources.

Trois pistes concrètes pour le mémoire

1. Un format intermédiaire réellement indépendant du langage cible
2. Une gestion des états d'interface (survol, clic, responsive)
3. Une prise en compte de l'accessibilité dès la construction de l'arbre

Références principales

- S. Xiao et al. *Prototype2Code*. arXiv :2405.04975, 2024.
- Y. Wan et al. *Divide-and-Conquer : Generating UI Code from Screenshots*. Proc. ACM Softw. Eng., FSE, 2025.
- Y. Jiang et al. *ScreenCoder*. arXiv :2507.22827, 2025.
- Y. Chen et al. *DesignCoder : Hierarchy-Aware and Self-Correcting UI Code Generation*. arXiv :2506.13663, 2025.